Ceph and software-defined storage

Understand and operate Ceph: RADOS, OSDs and monitors, CRUSH and placement groups, RBD, CephFS, RGW, Rook on Kubernetes, performance, failure recovery and capacity planning.

A MARGA printable pack. Free, offline, no sign-up. Learn online at marga.eknathalabs.com.

MARGA by EknathaLabs
marga.eknathalabs.com

Contents

1. What Ceph is: RADOS and the architecture (13 min)
2. Monitors, managers, OSDs and quorum (13 min)
3. CRUSH, pools and placement groups (13 min)
4. Deploying Ceph with cephadm (12 min)
5. RBD block storage for VMs and Kubernetes (13 min)
6. CephFS: shared file systems and the MDS (12 min)
7. RGW: S3 compatible object storage (12 min)
8. Rook: running Ceph on Kubernetes (13 min)
9. Replication, erasure coding and durability (13 min)
10. BlueStore, networks and Ceph performance (13 min)
11. Ceph failures, recovery and health warnings (13 min)
12. Capacity planning, upgrades and operating Ceph (13 min)
Cheat sheet

Lesson 1. What Ceph is: RADOS and the architecture

13 minutes. Understand Ceph as one distributed object store, RADOS, with block, file and object services on top, and learn what each daemon does.

Software-defined storage means the intelligence lives in software that runs on ordinary servers, not in a proprietary array. Ceph is the best known open source example, and it powers many OpenStack and Kubernetes clusters.

What you will learn

- What software-defined storage means
- RADOS and the three services on top
- The daemons and their jobs
- When Ceph is a good fit and when it is not

Software-defined storage

A traditional array puts disks, controllers and software in one box. A software-defined system runs on standard servers, keeps several copies of data across them, and heals itself when a disk or server fails. You scale by adding servers, and there is no single controller to fail.

RADOS and the services

RADOS is the reliable distributed object store at the bottom. Everything is stored as objects placed by an algorithm, not by a central table. Three services sit on top:

Service	What you get	Typical use
RBD	Block devices	VM disks, Kubernetes volumes, databases
CephFS	A POSIX shared file system	Home directories, shared data, RWX volumes
RGW	S3 and Swift object storage	Backups, artifacts, media

The daemons

Daemon	Count	Job
MON	3 or 5	Keep the cluster map and agree on it by quorum
MGR	2 or more	Metrics, dashboard and orchestration modules
OSD	One per disk	Store data, replicate, recover
MDS	2 or more per file system	Serve CephFS metadata
RGW	2 or more	Serve the S3 and Swift APIs

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: Ceph services on one object store

Part	What it does	What goes wrong
RBD block	Block devices for VMs and containers.	One slow OSD can slow many images.
CephFS	A shared POSIX file system.	An overloaded MDS stalls metadata operations.
RGW S3	An S3 compatible object gateway.	Large buckets need index sharding.
RADOS	The object store that places and protects data.	Everything above depends on it being healthy.
Monitors	Hold the cluster map and form a quorum.	Losing quorum stops the cluster.
OSDs on disks	Store objects and repair each other.	Too many down at once makes data unavailable.

Flow: RBD block to RADOS / CephFS to RADOS / RGW S3 to RADOS / RADOS to Monitors / RADOS to OSDs on disks

Fit

Ceph suits clusters of at least three nodes (preferably more) where you want one platform for several storage types and can operate it. It is a poor fit for a single server, a very small team with no time to learn it, or workloads that need the lowest possible latency from one local disk.

Incident: Three storage teams, one bill

Incident: *A company paid for three separate storage systems.*

```
$ ceph -s
  cluster:
    health: HEALTH_OK
  services:
    mon: 3 daemons, quorum a,b,c
    osd: 36 osds: 36 up, 36 in
    rgw: 2 daemons active
```

The consolidated cluster served VM disks, shared files and S3 buckets from 12 servers. The team learned one platform, added capacity by adding servers, and retired the old arrays as support contracts ended.

Common mistakes

- Running Ceph on a single node and calling it highly available
- Treating it as a drop in replacement for a local disk
- Ignoring the network, which carries all replication traffic
- Starting without anyone who will operate it

Check yourself

What is RADOS?

Answer: The object store under all Ceph services

Which daemon stores data?

Answer: OSD

What do monitors provide?

Answer: The cluster map and quorum

Which service gives block devices?

Answer: RBD

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 2. Monitors, managers, OSDs and quorum

13 minutes. Learn how monitors reach quorum, what managers and OSDs do, how BlueStore stores data, and how to read daemon status.

Four words explain most Ceph status output: quorum, up, down and in. Learn what each one means and what BlueStore does under an OSD.

What you will learn

- Quorum and why monitors come in odd numbers
- Manager and its modules
- OSD states: up, down, in and out
- BlueStore, the DB and WAL devices

Quorum

Monitors keep the authoritative cluster map: who the monitors and OSDs are, where they are, and how data is placed. They agree on changes by majority vote. With three monitors you can lose one; with five you can lose two. Two monitors tolerate nothing, since losing one leaves no majority. Without quorum clients can no longer find data.

Managers

The **manager** daemon collects metrics, runs the dashboard and Prometheus exporter, and hosts modules such as the PG autoscaler and the orchestrator. Run at least two: one active, one standby.

OSD states

An OSD has two independent states. **Up or down** says whether the daemon is running and talking. **In or out** says whether it takes part in data placement. A crashed OSD becomes down; after a timeout (10 minutes by default) it is marked out, and the cluster starts re-creating its data elsewhere.

Diagram: What happens when an OSD fails

Part	What it does	What goes wrong
OSD up and in	Serves reads and writes for its placement groups.	Hardware errors may appear before it fails.
OSD down	The daemon stopped answering; data is still on the disk.	Short outages heal quickly when it returns.
OSD out	After the timeout the cluster stops waiting for it.	Recovery now copies data to other OSDs.
Recovery and backfill	Other OSDs rebuild the missing copies.	Heavy recovery traffic can slow clients.

Flow: OSD up and in to OSD down / OSD down to OSD out / OSD out to Recovery and backfill / OSD down to OSD up and in

BlueStore

BlueStore manages the raw device itself. Object data goes to the main device; metadata goes to RocksDB; a write-ahead log makes writes safe. For hard drives, place the DB and WAL on an SSD or NVMe to speed up metadata and small writes. Size the DB at roughly 1 to 4 percent of the data device.

```
ceph -s
ceph osd tree
ceph osd stat
ceph mon stat
```

MARGA by EknathaLabs marga.eknathalabs.com

Incident: Lost quorum in one rack

Incident: *A power cut stopped all storage IO.*

```
$ ceph -s
  cluster:
    health: HEALTH_ERR
            1/3 mons down, quorum a
  services:
    mon: 3 daemons, quorum a (age 4m)
```

When the power dropped, two monitors failed together because they shared a rack. The one survivor could not form a majority. After power returned, the team spread monitors across three racks.

Common mistakes

- Running an even number of monitors
- Placing all monitors in one failure domain
- Confusing down with out
- Putting DB and WAL for many OSDs on one small SSD

Check yourself

How many monitors can a five monitor cluster lose?

Answer: Two

What does out mean for an OSD?

Answer: It no longer takes part in placement

What stores metadata inside BlueStore?

Answer: RocksDB

Why run two managers?

Answer: One active and one standby

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 3. CRUSH, pools and placement groups

13 minutes. Learn how Ceph decides where data lives using CRUSH, pools and placement groups, and how failure domains protect against losing a whole server or rack.

CRUSH is why Ceph scales. Clients compute where data lives instead of asking a central server, so there is no lookup bottleneck. The price is that you must understand failure domains.

What you will learn

- The path from object to OSD
- Failure domains and CRUSH rules
- Pools, replica count and min_size
- Placement group counts and the autoscaler

From object to OSD

1. An object is written into a **pool** under a name.
2. Its name is hashed to a **placement group** within the pool.
3. **CRUSH** maps that placement group to a list of OSDs, honouring the rule's failure domain.
4. The first OSD in the list is the primary; it forwards writes to the others.

Because CRUSH is a calculation over the cluster map, any client can do it and no one stores a per object table.

Diagram: Where does an object go?

Part	What it does	What goes wrong
Object	Has a name and data.	Very large objects can slow recovery.
Pool	Sets replica count, rule and PG number.	Wrong settings are costly to change later.
Placement group	A bucket of objects that moves as a unit.	Too few PGs gives uneven data spread.
CRUSH rule	Picks OSDs in different failure domains.	A rule on the wrong domain risks correlated loss.
OSDs on hosts	The disks that hold the copies.	Unequal weights cause hot spots.

Flow: Object to Pool / Pool to Placement group / Placement group to CRUSH rule / CRUSH rule to OSDs on hosts

Failure domains

A CRUSH rule says "choose N distinct buckets of type host" or rack, room or datacenter. With three replicas and host failure domain, each copy is on a different server. With rack, each is in a different rack and a whole rack can fail. You need enough buckets: three racks for three copies by rack.

Pools

```
ceph osd pool create vms 128
ceph osd pool set vms size 3
ceph osd pool set vms min_size 2
ceph osd pool application enable vms rbd
```

`size` is the number of copies. `min_size` is how many must be available to accept IO. With size 3 and min_size 2, the pool keeps working with one copy missing and stops at two missing, protecting against writing data that exists only once.

Placement groups

Each OSD should carry about 100 PGs. Too few makes data uneven; too many wastes memory. Leave the **PG autoscaler** on, and set a `target_size_ratio` for pools you know will grow.

MARGA by EknathaLabs marga.eknathalabs.com

Incident: Two copies on one host

Incident: *Losing a server took some data offline.*

```
$ ceph osd crush rule dump replicated_rule | grep type
                "type": "osd"
$ ceph health detail
HEALTH_ERR 12 pgs inactive; 12 pgs stuck
```

The pool used a rule with an OSD failure domain, so two of three copies of some PGs shared a server. When that server failed, the PGs dropped below min_size. A host failure domain rule fixed the placement and data moved apart.

Common mistakes

- Leaving the default rule without checking its failure domain
- Setting min_size to 1
- Creating pools with a fixed PG count and never revisiting it
- Using a rack failure domain with fewer racks than copies

Check yourself

What decides which OSDs hold a placement group?

Answer: CRUSH

What does min_size control?

Answer: How many copies must be up to accept IO

Why choose host as the failure domain?

Answer: Copies land on different servers

What does the PG autoscaler do?

Answer: Adjusts PG counts as pools grow

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 4. Deploying Ceph with cephadm

12 minutes. Bootstrap a cluster with cephadm, add hosts and OSDs, deploy services with the orchestrator, and read cluster status.

Cephadm is the standard way to deploy Ceph on bare metal or VMs. It needs a container runtime, systemd, time sync and Python, and gives you one command to bootstrap and an orchestrator for everything after.

What you will learn

- Prerequisites for a healthy first cluster
- Bootstrapping the first host
- Adding hosts, OSDs and services
- Reading status and finding what is wrong

Prerequisites

Each host needs a supported Linux, systemd, a container runtime (Podman or Docker), LVM2, Python 3, time synchronisation and DNS or hosts entries. Use a separate network for cluster traffic if you can. Disks for OSDs should be empty with no partitions or file systems.

Bootstrap

```
cephadm bootstrap --mon-ip 10.0.0.11
ceph orch host add ceph2 10.0.0.12
ceph orch host add ceph3 10.0.0.13
ceph orch apply osd --all-available-devices
ceph orch ls
```

Bootstrap creates the first monitor and manager, a dashboard, and an admin keyring. The orchestrator then places daemons on hosts you add.

Diagram: How cephadm builds a cluster

Part	What it does	What goes wrong
cephadm bootstrap	Creates the first MON and MGR and the admin keyring.	Use the right monitor IP; it is hard to change later.
Orchestrator	Runs in the manager and places daemons.	If the manager is down, orchestration pauses.
Added hosts	Receive monitors, managers and OSDs by label.	Wrong labels place daemons in one rack.
OSDs on free disks	Created from empty devices.	A disk with old data is skipped.
Service specification	A YAML file that describes the wanted daemons.	An unreviewed spec can change production.

Flow: cephadm bootstrap to Orchestrator / Service specification to Orchestrator / Orchestrator to Added hosts / Orchestrator to OSDs on free disks

MARGA by EknathaLabs marga.eknathalabs.com

Service specifications

```
service_type: mon
placement:
  count: 5
  label: mon
---
service_type: osd
service_id: hdd_with_nvme_db
placement:
  host_pattern: 'osd*'
spec:
  data_devices:
    rotational: 1
  db_devices:
    rotational: 0
```

Apply with `ceph orch apply -i spec.yaml`. A drive group like this puts the OSD data on hard drives and their DB on flash.

Reading status

`ceph -s` gives health and services. `ceph orch ps` lists daemons and their state. `ceph health detail` explains each warning. `cephadm logs --name osd.3` shows a daemon's log.

Incident: The disk that was skipped

Incident: *A new disk never became an OSD.*

```
$ ceph orch device ls
HOST   PATH      AVAILABLE  REJECT REASONS
osd4   /dev/sdc  No         Has partitions, LVM detected
$ ceph orch device zap osd4 /dev/sdc --force
zap successful for /dev/sdc on osd4
```

The disk had an old partition table from a previous use, so cephadm refused it to protect data. The team confirmed the disk was unused, zapped it, and the orchestrator created the OSD automatically.

Common mistakes

- Skipping time synchronisation on hosts
- Running ad hoc commands that are not recorded
- Zapping a disk without checking what is on it
- Putting all monitors on the same switch

Check yourself

What does cephadm run each daemon in?

Answer: A container

What does ceph orch apply osd --all-available-devices do?

Answer: Creates OSDs on empty disks

Why keep a spec in Git?

Answer: To rebuild the same cluster

Which command explains warnings?

Answer: ceph health detail

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 5. RBD block storage for VMs and Kubernetes

13 minutes. Create and use RBD images, snapshots and clones, connect Ceph to Kubernetes with ceph-csi, and replicate images between clusters.

Most Ceph users meet RBD first. It gives you thin provisioned, resizable disks with snapshots and clones, spread over every OSD so no single disk limits performance.

What you will learn

- Creating and using images
- Snapshots, clones and layering
- Using RBD with Kubernetes and VMs
- Mirroring for disaster recovery

Images

```
ceph osd pool create rbd 64
rbd pool init rbd
rbd create rbd/db1 --size 100G
rbd info rbd/db1
rbd map rbd/db1
rbd du rbd/db1
```

Images are thin provisioned: they use space only as data is written. They are striped over many objects, so reads and writes hit many OSDs in parallel. Resize with `rbd resize` and grow the file system afterwards.

Snapshots and clones

A snapshot is a read-only point in time. A **clone** is a writable child of a protected snapshot that shares unchanged data. That makes VM provisioning fast: keep a golden image snapshot and clone it per VM. Flatten a clone to copy the data and remove the dependency on the parent.

Diagram: RBD in a Kubernetes cluster

Part	What it does	What goes wrong
Pod or VM	Uses a PVC as a disk.	Heavy IO from one pod can affect others.
ceph-csi	Creates, maps and mounts RBD images for PVCs.	Wrong credentials leave volumes Pending.
RBD image	Thin provisioned and striped across OSDs.	Unremoved snapshots grow the pool.
Replicated pool	Holds the objects with three copies.	A full pool stops all images in it.
Mirror to site B	Copies images to another cluster.	Mirroring lag sets the data lost in a disaster.

Flow: Pod or VM to ceph-csi / ceph-csi to RBD image / RBD image to Replicated pool / RBD image to Mirror to site B

Kubernetes and VMs

A StorageClass using the `rbd.csi.ceph.com` provisioner turns a PVC into an RBD image. Use `volumeMode: Block` with `ReadWriteMany` for VMs that need live migration, and ext4 or XFS with ReadWriteOnce for databases. QEMU also talks to RBD directly through librbd.

Mirroring

Journal based mirroring replicates every write; snapshot based mirroring copies at intervals. Choose by how much data loss you can accept, and test a promote of the secondary site.

Incident: The pool that filled with snapshots

Incident: *A pool reached nearfull with little live data.*

MARGA by EknathaLabs marga.eknathalabs.com

```
$ rbd du rbd/db1
NAME           PROVISIONED  USED
db1@pre-up-1   100 GiB      18 GiB
db1@pre-up-2   100 GiB      21 GiB
db1            100 GiB      64 GiB
<TOTAL>        100 GiB      1.9 TiB
```

Upgrade snapshots had accumulated for a year and each pinned changed blocks. A cleanup job that deleted snapshots older than a week returned over a terabyte of space.

Common mistakes

- Keeping snapshots forever
- Resizing the image without growing the file system
- Using one pool for everything
- Skipping a test of the mirror promote

Check yourself

What is an RBD clone?

Answer: A writable child of a snapshot

What does ceph-csi do?

Answer: Provides Ceph volumes to Kubernetes

Why are RBD images fast?

Answer: They are striped across many OSDs

What does RBD mirroring give?

Answer: A copy in another cluster

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 6. CephFS: shared file systems and the MDS

12 minutes. Run CephFS for shared POSIX storage, understand how metadata servers work, mount it from clients and Kubernetes, and set quotas and snapshots.

CephFS fills the gap between block and object: a POSIX file system many hosts can mount at the same time. It is built on the same RADOS, with metadata servers added.

What you will learn

- Data and metadata pools
- Active and standby MDS
- Mounting, subvolumes and quotas
- Using CephFS for Kubernetes ReadWriteMany

Data and metadata

File contents are stored as objects in a **data pool**. Directory structure and file attributes live in a **metadata pool**, served from memory by **MDS** daemons. Keep the metadata pool on fast media.

```
ceph fs volume create shared
ceph fs status shared
ceph fs subvolume create shared proj-a --size 107374182400
```

Active and standby MDS

An active MDS holds part of the namespace in cache. Standby MDS daemons take over if it fails, usually in seconds. With multiple active MDS (`max_mds`), directories are split between them to spread load. The MDS cache size (`mds_cache_memory_limit`) decides how many files it can keep hot.

Diagram: How a CephFS client reads a file

Part	What it does	What goes wrong
Client mount	A kernel or FUSE mount of the file system.	Old client kernels may lack features.
Active MDS	Answers path, permission and layout questions.	A cold or small cache makes ls slow.
Standby MDS	Waits to take over a failed MDS.	A slow takeover stalls clients.
Metadata pool	Stores directory and inode data.	Place it on SSD or NVMe.
Data pool	Stores file contents as objects.	Wrong replication here risks data.

Flow: Client mount to Active MDS / Active MDS to Metadata pool / Client mount to Data pool / Standby MDS to Active MDS

The client asks the MDS where a file's data is, then reads and writes the data pool directly. File data does not pass through the MDS.

Mounting and quotas

```
mount -t ceph 10.0.0.11:6789:/ /mnt/shared -o name=app,secretfile=/etc/ceph/app.secret
```

Subvolumes isolate projects, carry quotas and can be snapshotted. Create one per tenant or application and mount only that path with its own key.

Kubernetes

ceph-csi provisions CephFS subvolumes as ReadWriteMany PVCs, handy for shared uploads and content. For databases, prefer RBD.

Incident: Slow ls in a huge directory

Incident: *Listing a directory with millions of files took minutes.*

MARGA by EknathaLabs marga.eknathalabs.com

```
$ ceph fs status shared
RANK  STATE   MDS    ACTIVITY     DNS   INOS
 0    active  mds.a  Reqs: 9800/s 4.1M  4.1M
$ ceph tell mds.a perf dump | grep cache
    "mds_mem.ino": 4100000
```

One active MDS handled every metadata request and had a cache too small for four million files. The team raised max_mds, increased the cache limit, and changed the application to spread files over subdirectories.

Common mistakes

- Putting the metadata pool on slow disks
- Using CephFS for databases
- One huge shared directory for all users
- Mounting the whole file system for every application

Check yourself

What does the MDS serve?

Answer: File system metadata

Where is CephFS file data stored?

Answer: In a data pool

What takes over when an MDS fails?

Answer: A standby MDS

What is a subvolume for?

Answer: Isolating a project with a quota

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 7. RGW: S3 compatible object storage

12 minutes. Run the RADOS Gateway for S3 and Swift, create users and buckets, understand realms, zones and multisite replication, and manage bucket lifecycles.

With RGW you can offer an S3 compatible service from your own data centre. It is common for backups, artifacts, data lakes and applications that must keep data on site.

What you will learn

- How RGW stores objects
- Users, keys, buckets and policies
- Index sharding and lifecycle
- Multisite replication

How it works

RGW is a stateless HTTP gateway. It translates S3 requests into RADOS operations on several pools: one for object data, others for bucket indexes, logs and metadata. Run at least two RGW daemons behind a load balancer such as HAProxy and terminate TLS there or in RGW.

```
radosgw-admin user create --uid=backup --display-name="Backup app"
aws --endpoint-url https://s3.example.com s3 mb s3://nightly
aws --endpoint-url https://s3.example.com s3 cp db.dump s3://nightly/
```

Users and policies

Each application gets its own user, access key and secret key, with quotas. Bucket policies and IAM style policies control access. Never share the admin user's keys with applications.

Diagram: S3 requests through RGW

Part	What it does	What goes wrong
S3 client	Signs requests with an access key.	Keys in code leak through repositories.
Load balancer + TLS	Spreads requests and ends HTTPS.	A single balancer is a single point of failure.
RGW daemons	Translate S3 into RADOS operations.	Too few daemons limit throughput.
Bucket index pool	Maps object names to locations.	Huge buckets need index shards.
Data pool	Holds object data, often erasure coded.	Small objects waste space on EC.

Flow: S3 client to Load balancer + TLS / Load balancer + TLS to RGW daemons / RGW daemons to Bucket index pool / RGW daemons to Data pool

Index sharding and lifecycle

Listing a bucket reads its index. A bucket with millions of objects needs several index shards, and dynamic resharding adds them automatically. Lifecycle rules expire or transition objects by age, which keeps temporary data from filling the cluster.

Multisite

A **realm** contains **zone groups**, which contain **zones**. Each zone is its own cluster. Metadata and data replicate between zones asynchronously, so a second site holds a copy for disaster recovery or local reads. Replication is eventually consistent, so applications must tolerate a short lag.

Incident: A bucket too big to list

Incident: *Listing a bucket timed out after months of backups.*

```
$ radosgw-admin bucket limit check
"bucket": "nightly", "num_objects": 410000000, "num_shards": 11, "fill_status": "OVER 100%"
$ radosgw-admin bucket reshard add --bucket=nightly --num-shards=211
```

MARGA by EknathaLabs marga.eknathalabs.com

```
queued for resharding
```

The bucket index had far too few shards for the number of objects, so listing and writes slowed. Resharding it to more shards, and turning on dynamic resharding, restored normal speed.

Common mistakes

- Sharing the admin credentials
- One RGW daemon with no load balancer
- Never setting lifecycle rules on temporary buckets
- Expecting multisite to be instant

Check yourself

What does RGW provide?

Answer: S3 and Swift APIs

What does a zone group contain?

Answer: Zones

Why shard a bucket index?

Answer: To keep listing and writes fast

How does multisite replicate?

Answer: Asynchronously between zones

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 8. Rook: running Ceph on Kubernetes

13 minutes. Deploy and operate Ceph inside Kubernetes with the Rook operator, define clusters, pools and storage classes as custom resources, and use the toolbox for ceph commands.

Running Ceph inside Kubernetes is attractive: one platform, declarative storage, and the operator replaces daemons that fail. It also couples your storage lifecycle to cluster operations, which needs care.

What you will learn

- What the Rook operator manages
- Defining the cluster and its pools
- Hyperconverged and dedicated storage nodes
- Day two: toolbox, upgrades and safe node maintenance

The operator

Rook runs as a Deployment. You create a `CephCluster` custom resource and it starts monitors, managers and OSD pods, one OSD per device. If a pod dies, Kubernetes and the operator restart it. Rook also installs ceph-csi so PVCs can use the pools.

```
apiVersion: ceph.rook.io/v1
kind: CephCluster
metadata:
  name: rook-ceph
  namespace: rook-ceph
spec:
  cephVersion:
    image: quay.io/ceph/ceph:v18
  dataDirHostPath: /var/lib/rook
  mon:
    count: 3
    allowMultiplePerNode: false
  storage:
    useAllNodes: false
    useAllDevices: false
    nodes:
      - name: storage1
        devices:
          - name: nvme1n1
```

Pools and classes

```
apiVersion: ceph.rook.io/v1
kind: CephBlockPool
metadata:
  name: replicapool
  namespace: rook-ceph
spec:
  failureDomain: host
  replicated:
    size: 3
```

A StorageClass then points at this pool. PVCs from it become RBD images.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: Rook turning a custom resource into storage

Part	What it does	What goes wrong
CephCluster resource	Declares monitors, devices and the Ceph version.	A wrong device list can wipe a disk.
Rook operator	Creates and heals the Ceph pods.	If the operator is down nothing is repaired.
MON, MGR and OSD pods	The running Ceph daemons.	Pods on one node share a failure.
StorageClass + CSI	Turns PVCs into RBD or CephFS volumes.	Wrong pool names leave PVCs Pending.
Application PVC	Uses the storage like any volume.	Heavy writers need their own pool or class.

Flow: CephCluster resource to Rook operator / Rook operator to MON, MGR and OSD pods / Rook operator to StorageClass + CSI / StorageClass + CSI to Application PVC

Where to run it

Hyperconverged means storage and applications share nodes, which saves hardware but couples CPU and memory use. Dedicated storage nodes, selected with labels and taints, give steadier performance. Either way, OSD pods need local disks or PVCs from a fast class.

Day two

The **toolbox** pod gives you the `ceph` command: `kubectl -n rook-ceph exec -it deploy/rook-ceph-tools -- ceph -s`. Upgrade the operator first, then the Ceph image. Before draining a node, check that the cluster is healthy and that removing the node's OSDs keeps enough replicas.

Incident: The drain that took data offline

Incident: *A node drain started a cascade of warnings.*

```
$ kubectl -n rook-ceph exec deploy/rook-ceph-tools -- ceph -s
    health: HEALTH_WARN
            Degraded data redundancy: 412/9120 objects degraded
$ kubectl get nodes
storage2   Ready,SchedulingDisabled
```

A second node was drained before recovery from the first finished, so some data had one copy. The team now waits for HEALTH_OK between drains and uses Pod Disruption Budgets that Rook creates.

Common mistakes

- Draining a second storage node before the cluster is healthy again
- Using useAllDevices on nodes with data disks
- Mixing storage and heavy application load without limits
- Upgrading Ceph before the operator

Check yourself

What does the Rook operator watch?

Answer: Custom resources such as CephCluster

How do you run ceph commands in Rook?

Answer: Through the toolbox pod

What does a CephBlockPool define?

Answer: A pool and its replica settings

What should you check before draining a storage node?

Answer: That the cluster is HEALTH_OK

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 9. Replication, erasure coding and durability

13 minutes. Compare replicated and erasure coded pools, work out the capacity overhead and failure tolerance of each, and choose a layout that fits the workload.

Capacity and durability pull in different directions. Ceph lets you pick per pool, so you can match the protection to what the data is worth and how it is accessed.

What you will learn

- Replicated pools
- Erasure coding with k and m
- Capacity overhead and tolerance
- Choosing a layout for the workload

Replication

Replicated pools keep `size` full copies on different failure domains. They are simple and fast, repair quickly by copying one replica, and handle small random writes well. The cost is raw space: three copies use three times the data size.

Erasure coding

An EC profile splits each object into **k** data chunks and **m** coding chunks placed on different failure domains. Any k of the k+m chunks can rebuild the object, so the pool survives m failures.

Layout	Raw space per usable	Survives	Notes
Replicated size 3	3.0x	2 lost copies	Fast, simple
EC 4+2	1.5x	2 lost chunks	Needs at least 6 hosts
EC 8+3	1.375x	3 lost chunks	Needs at least 11 hosts

```
ceph osd erasure-code-profile set ec42 k=4 m=2 crush-failure-domain=host
ceph osd pool create archive 64 64 erasure ec42
ceph osd pool set archive allow_ec_overwrites true
```

Diagram: Three copies versus four plus two

Part	What it does	What goes wrong
12 TB of data	The amount users store.	Plan raw capacity from this number.
Replicated size 3	36 TB raw, fast and simple.	Expensive for large cold data.
Erasure coded 4+2	18 TB raw with two parity chunks.	Slower small writes and heavier recovery.
Two failures	Both layouts survive losing two failure domains.	A third loss in either layout means data loss.

Flow: 12 TB of data to Replicated size 3 / 12 TB of data to Erasure coded 4+2 / Replicated size 3 to Two failures / Erasure coded 4+2 to Two failures

Choosing

Use replicated pools for RBD databases, VM disks and CephFS metadata. Use erasure coding for RGW data, backups and archives, where objects are large and written once. Block devices on EC need `allow_ec_overwrites` and a replicated pool for metadata, and still suit sequential rather than random IO.

MARGA by EknathaLabs marga.eknathalabs.com

Durability is more than copies

Durability also depends on failure domains, how fast recovery runs, scrubbing that finds bit rot, and spare capacity to rebuild into. A cluster 95 percent full cannot recover from a failure.

Incident: Not enough hosts for the profile

Incident: *Creating an EC pool left PGs stuck.*

```
$ ceph health detail
HEALTH_WARN Reduced data availability: 64 pgs inactive
pg 7.3 is creating+incomplete, acting [4,9,NONE,14,2,NONE]
$ ceph osd tree | grep -c host
5
```

A 4+2 profile with host failure domain needs six hosts. With five hosts, two chunks of each PG had nowhere to go. The team added a sixth host, and the PGs became active.

Common mistakes

- Using EC for small random write workloads
- Choosing k and m with fewer hosts than k+m
- Counting raw space instead of usable space
- Running above 80 percent full and expecting to recover

Check yourself

How much raw space does size 3 replication use?

Answer: Three times the data

What does EC 4+2 tolerate?

Answer: Two lost chunks

Where does EC fit best?

Answer: Large, write-once objects

Why keep free capacity?

Answer: Recovery needs room to rebuild

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 10. BlueStore, networks and Ceph performance

13 minutes. Find and fix Ceph performance limits: media choices, DB and WAL placement, public and cluster networks, benchmarks with rados and rbd, and recovery throttling.

A Ceph write travels from the client to a primary OSD, then to its replicas over the network, and is acknowledged only when all have it. Anything slow along that path slows the write.

What you will learn

- Choosing media and where the DB and WAL go
- Public and cluster networks
- Benchmarking before and after a change
- Keeping recovery from hurting clients

Media

NVMe gives the best latency for RBD databases. SSD suits general VM storage. Hard drives suit capacity for object data, with DB and WAL on flash. Use enterprise drives with power loss protection, because Ceph asks drives to flush data safely. Plan memory: the default `osd_memory_target` is 4 GiB per OSD.

Networks

Client traffic uses the **public network**. Replication, recovery and heartbeats can use a separate **cluster network**. Because each client write becomes two further replica writes, cluster traffic is roughly twice the client load, plus recovery. Use at least 25 GbE for dense nodes, and make sure MTU matches on every switch if you use jumbo frames.

Diagram: A replicated write

Part	What it does	What goes wrong
Client	Sends the write to the primary OSD.	A slow client link caps throughput.
Primary OSD	Writes locally and forwards to replicas.	One slow disk delays every write that uses it.
Replica OSD 2	Writes the copy and acknowledges.	Cluster network congestion slows it.
Replica OSD 3	Writes the copy and acknowledges.	A busy recovery competes for the same disk.
Acknowledge	Sent to the client after all copies are safe.	Latency is the slowest replica.

Flow: Client to Primary OSD / Primary OSD to Replica OSD 2 / Primary OSD to Replica OSD 3 / Replica OSD 2 to Acknowledge / Replica OSD 3 to Acknowledge

Benchmarks

```
rados bench -p testpool 30 write --no-cleanup
rados bench -p testpool 30 seq
rbd bench --io-type write --io-size 4K --io-threads 16 rbd/test
fio --name=t --filename=/dev/rbd0 --rw=randwrite --bs=4k --iodepth=32 --runtime=60 --direct=1
```

Run the same test before and after each change, and run it from more than one client. A single client may hit its own limit.

Recovery and scrub

Recovery and backfill compete with clients. The **mClock** scheduler lets you choose a profile such as `high_client_ops`. Schedule scrubs outside business hours.

MARGA by EknathaLabs marga.eknathalabs.com

Incident: Latency spikes after every disk swap

Incident: *Replacing a failed disk made VMs slow for hours.*

```
$ ceph -s | grep -E 'recovery|client'
    client:   220 MiB/s rd, 90 MiB/s wr
    recovery: 780 MiB/s, 190 objects/s
$ ceph config get osd osd_mclock_profile
high_recovery_ops
```

The cluster had been left on the high_recovery_ops profile after an earlier rebuild, so recovery used most of the disk bandwidth. Switching to balanced kept clients responsive while data healed.

Common mistakes

- Using consumer SSDs without power loss protection
- Changing several settings at once
- Benchmarking from one client only
- Leaving recovery at maximum priority

Check yourself

Where does client traffic flow?

Answer: The public network

When is a replicated write acknowledged?

Answer: After all copies are written

What does mClock help with?

Answer: Balancing client and recovery IO

What should DB and WAL go on for HDD OSDs?

Answer: Flash

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 11. Ceph failures, recovery and health warnings

13 minutes. Read Ceph health output, handle failed disks and servers, understand backfill and recovery, and fix nearfull, slow ops, clock skew and inconsistent placement groups.

In Ceph, status is information. Most warnings are the cluster telling you it is healing. A few mean data is at risk or writes are about to stop. Learn to tell them apart.

What you will learn

- Reading ceph -s and health detail
- Disk and host failures
- Planned maintenance with noout
- Capacity thresholds, slow ops and inconsistent PGs

Where to look

```
ceph -s
ceph health detail
ceph osd tree
ceph pg stat
ceph osd df
```

`ceph -s` shows health, daemons and PG states. `ceph health detail` explains each warning with the affected daemons. Healthy PGs are `active+clean`. States such as `degraded`, `undersized`, `backfilling` and `remapped` show work in progress. `inactive` or `incomplete` means IO is blocked and is urgent.

Failures

When an OSD stops, its PGs run with fewer copies (degraded) but keep serving IO as long as min_size is met. After the down-to-out timeout, Ceph rebuilds the missing copies on other OSDs. If you replace a failed disk, remove the old OSD and create a new one; data flows back by backfill.

Diagram: Handling a failed disk

Part	What it does	What goes wrong
OSD goes down	Its PGs are degraded but still served.	Two failures in the same PG are the danger.
Marked out after timeout	The cluster starts rebuilding copies.	Recovery load can slow clients.
Replace the disk	Remove the old OSD and add a new one.	Zap the right disk.
Backfill	Data flows to the new OSD.	Watch the progress in ceph -s.
active+clean	All PGs have full copies again.	Only then move to the next change.

Flow: OSD goes down to Marked out after timeout / Marked out after timeout to Backfill / OSD goes down to Replace the disk / Replace the disk to Backfill / Backfill to active+clean

Planned maintenance

Run `ceph osd set noout` before rebooting a host so its OSDs are not marked out and rebuilt unnecessarily. Reboot one host, wait for the OSDs to rejoin and PGs to return to clean, then `ceph osd unset noout`.

MARGA by EknathaLabs marga.eknathalabs.com

Common warnings

Warning	Meaning	Action
nearfull (85%)	An OSD is almost full	Add capacity or delete data now
backfillfull (90%)	Backfill to that OSD stops	Free space before more failures
full (95%)	Writes are blocked	Emergency: add space or delete
slow ops	Requests wait on slow OSDs or network	Check disks and network
clock skew	Monitors disagree on time	Fix NTP or chrony
inconsistent PG	Scrub found a mismatch	Investigate, then `ceph pg repair`

Incident: Three reboots in a row

Incident: *Rebooting hosts one after another made data unavailable.*

```
$ ceph pg stat
2048 pgs: 188 active+undersized+degraded, 31 down, 1829 active+clean
$ ceph health detail | head -n 3
HEALTH_ERR Reduced data availability: 31 pgs inactive, 31 pgs down
```

The operator rebooted the second and third hosts before recovery from the first had finished. Some PGs then had only one copy, below min_size. Bringing a host back restored IO. The team wrote a rule: set noout, one host at a time, wait for HEALTH_OK.

Common mistakes

- Rebooting several storage hosts at once
- Ignoring nearfull until the cluster is full
- Running repair before understanding why a PG is inconsistent
- Forgetting to unset noout

Check yourself

What does active+clean mean?

Answer: The PG is healthy

What does noout do?

Answer: Stops OSDs being marked out during maintenance

What happens at the full threshold?

Answer: Writes are blocked

What causes a clock skew warning?

Answer: Monitors disagree on time

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 12. Capacity planning, upgrades and operating Ceph

13 minutes. Plan raw and usable capacity with headroom for recovery, upgrade a cluster safely in the right order, expand it, and build a day-two routine with monitoring and runbooks.

The first deployment is the easy part. Day two is capacity forecasts, upgrades, hardware replacement and the habits that keep a storage cluster boring.

What you will learn

- Raw, usable and safe capacity
- Upgrading in the right order
- Expanding the cluster
- Monitoring and a day-two routine

Capacity

Usable capacity is raw capacity divided by the replication overhead, minus headroom. For three-way replication, 300 TB raw gives at most 100 TB usable. The cluster must also survive losing a failure domain: if one host of ten fails, the other nine must absorb its data. That means staying well below 90 percent. A common target is 70 to 75 percent used, with alerts at 65 and 75.

```
ceph df
ceph osd df tree
ceph osd pool autoscale-status
```

Watch uneven OSD usage in `ceph osd df`. The **balancer** module moves PGs to even it out.

Upgrades

```
ceph orch upgrade start --ceph-version 18.2.4
ceph orch upgrade status
```

cephadm upgrades monitors first, then managers, then OSDs, then MDS and RGW, one at a time, waiting for the cluster to be healthy between steps. Read the release notes, upgrade one major version at a time, and rehearse in a test cluster.

Diagram: A safe change routine

Part	What it does	What goes wrong
Check health	Cluster is HEALTH_OK before you start.	Starting on a warning hides new problems.
Make one change	An upgrade step, a new host or a disk swap.	Several changes at once make faults hard to trace.
Watch recovery	PGs return to active+clean.	Client latency may rise during rebalancing.
Next change	Only after everything is clean.	Skipping the wait is how outages start.
Capacity forecast	Tracks growth and the failure headroom.	Buying hardware late forces rushed changes.

Flow: Check health to Make one change / Make one change to Watch recovery / Watch recovery to Next change / Capacity forecast to Make one change

Expanding

Add hosts and OSDs in steps. New OSDs trigger rebalancing, so add capacity before the cluster is full, and consider setting the upmap balancer so data moves gradually. Replacing old hardware follows the same path: add new, drain old, wait for clean.

MARGA by EknathaLabs marga.eknathalabs.com

Routine

Alerts on health state, capacity thresholds, slow ops, scrub errors, OSD flapping and clock skew. A written runbook for each. Quarterly: test a restore, a disk swap and an upgrade in a test cluster.

Incident: Full because a host failed

Incident: *Writes stopped after a hardware failure.*

```
$ ceph df
--- RAW STORAGE ---
TOTAL  300 TiB  34 TiB  266 TiB   88.7
$ ceph health
HEALTH_ERR 3 full osd(s); pool(s) full
```

At 82 percent used the cluster had no room to rebuild a lost host. Recovery filled the surviving OSDs and writes stopped. The team added hosts, then set capacity alerts at 65 and 75 percent.

Common mistakes

- Planning capacity for the healthy day only
- Upgrading several components at once
- Adding hardware only when the cluster is nearly full
- No alerts on capacity trends

Check yourself

How much usable space does 300 TB raw give at size 3?

Answer: About 100 TB

What upgrades first with cephadm?

Answer: Monitors

Why keep headroom?

Answer: A failed host's data must be rebuilt

What does the balancer do?

Answer: Evens out PG placement

MARGA by EknathaLabs
marga.eknathalabs.com

Cheat sheet: Ceph

Daemons, status commands, pools, placement groups, cephadm, RBD, health warnings and a safe maintenance routine.

Status

```
ceph -s
ceph health detail
ceph osd tree
ceph osd df tree
ceph df
ceph pg stat
```

Warning: *Healthy placement groups are* `active+clean`*.* `inactive` *or* `incomplete` *means IO is blocked and is urgent.*

Daemons

Daemon	Job
MON	Cluster map and quorum, run 3 or 5
MGR	Metrics, dashboard, modules
OSD	Stores data on one disk
MDS	CephFS metadata
RGW	S3 and Swift gateway

Pools

```
ceph osd pool create vms 128
ceph osd pool set vms size 3
ceph osd pool set vms min_size 2
ceph osd pool application enable vms rbd
ceph osd pool autoscale-status
```

Danger: *Never set* `min_size 1`*. A pool can then accept writes that exist as a single copy.*

cephadm

```
cephadm bootstrap --mon-ip 10.0.0.11
ceph orch host add ceph2 10.0.0.12
ceph orch apply osd --all-available-devices
ceph orch ps
ceph orch upgrade start --ceph-version 18.2.4
```

Warning: *Zap a disk only after you confirm it is spare.* `ceph orch device zap` *destroys its data.*

RBD

```
rbd create rbd/db1 --size 100G
rbd du rbd/db1
rbd snap create rbd/db1@before-upgrade
rbd snap ls rbd/db1
```

Warning: *Old snapshots keep growing. Delete them on a schedule and watch* `rbd du`*.*

Maintenance

```
ceph osd set noout
ceph osd unset noout
ceph pg repair 7.3
```

Danger: *Reboot one host at a time and wait for HEALTH_OK. Several at once can drop PGs below min_size.*

MARGA by EknathaLabs
marga.eknathalabs.com

Capacity thresholds

Level	Meaning
85% nearfull	Warning, act now
90% backfillfull	Backfill to that OSD stops
95% full	Writes are blocked

Quick health sweep

```
ceph -s
ceph health detail
ceph osd tree
ceph df
ceph osd pool ls detail
```

MARGA by EknathaLabs
marga.eknathalabs.com